

Name:- Shardul Narvekar Roll no: VAI05BT22414

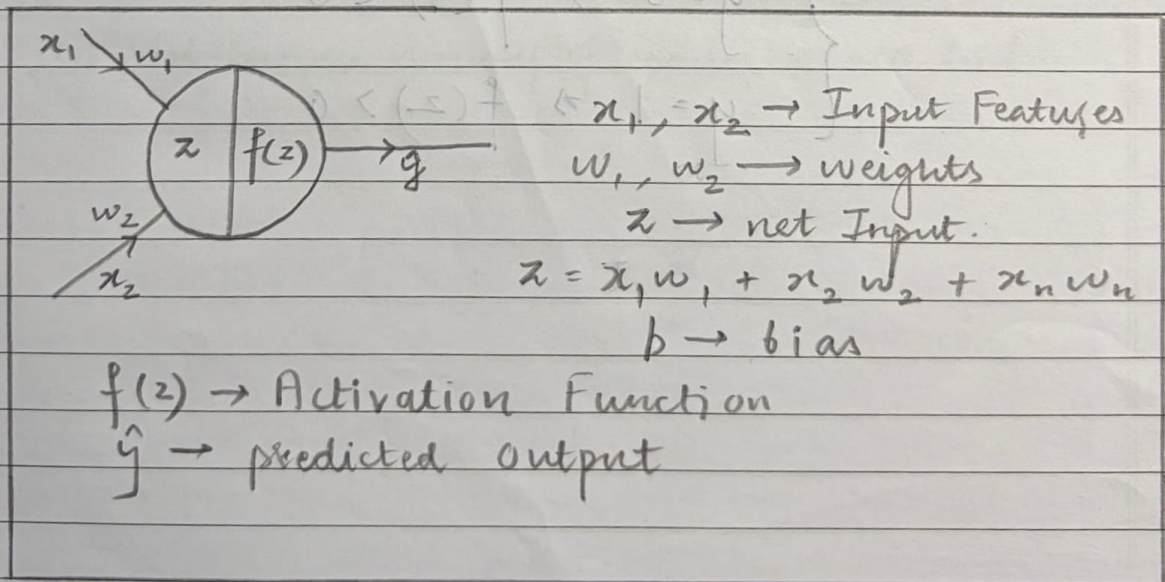
Div: B Subject: Deep Learning.

1) What is a Perceptron?

Ans: A Perceptron is the simplest type of artificial neural network used for binary classification. A single perceptron can classify only linearly separable data. It consists of input values, weights, a bias, and an activation function that together determine the output. The perceptron learns by adjusting its weights based on errors during training. Nonlinearly separable data requires a multi-layer perceptron with hidden layers and nonlinear activation functions.

2) Architecture of a Perceptron?

Ans:-



3) Mathematical model

The mathematical model of a perceptron considers the weighted sum of the inputs and adds a bias. The result is then passed through an activation function to produce the final output. (predicted)

$$y = f\left(\sum_{i=1}^n w_i x_i + b\right); \quad z = \sum_{i=1}^n w_i x_i + b$$

Activation functions is decided by the programmer eg:- step-function, Sigmoid, ReLU, Tanh

x_i = input values; w_i = weights; b = Bias; y = Output
 f = Activation function.

(1958)
Step Function by Frank Rosenblatt is the most basic ~~for~~ Activation Function.

$$\begin{cases} y = 0 \Rightarrow f(z) \leq 0 \\ y = 1 \Rightarrow f(z) > 0 \end{cases}$$

4) Learning Rule

Ans:- The perceptron Learning Rule updates the weights and bias based on the prediction error. If the predicted output is incorrect, the weights and bias are adjusted to reduce the error.

Weight Update:-

$$w_{\text{new}} = w_{\text{old}} + \eta (y_{\text{actual}} - y_{\text{predicted}}) x$$

Bias Update

$$b_{\text{new}} = b_{\text{old}} + \eta (y_{\text{actual}} - y_{\text{predicted}})$$

where:-

η (eta) $\Rightarrow$ Learning Rate ; y_{actual} $\Rightarrow$ Actual output value;
 x $\Rightarrow$ Input value (vector) ; $y_{\text{predicted}}$ $\Rightarrow$ Predicted output value.

$\Rightarrow$ The training data is processed repeatedly over multiple epochs. In each epoch, the perceptron updates its weights and bias until the classification error is minimised or the maximum number of epochs is reached.

5) Algorithm of a Single Perceptron

Ans:-

Step 1 $\Rightarrow$ Initialize the weights. Import the required Library (Numpy) for Numerical Computations.

Step 2 $\Rightarrow$ Store the input features and target output in separate variables. Initialize the weights and bias with small values or zeros.

Step 3 $\Rightarrow$ Define activation function (step function) to produce binary output (0, 1). Set learning rate (η) and no. of epochs.

Step 4 $\Rightarrow$ For each epoch:- $f(z) \Rightarrow z = \sum_{i=1}^n w_i x_i + b$

Step 5 $\Rightarrow$ Compute the error: Actual value - Predicted value.

Step 6 $\Rightarrow$ Update the weights and bias using Perceptron Learning Rule.

Step 7:- Repeat step 4, 5, 6 until the error becomes zero or the maximum number of epochs is reached.

Step 8:- Display the final weight, bias, and predictions for the given input data.

Name: Shardul Narvekar

Roll No.: UAI05BT223414

Division: B

Subject: Deep Learning

Q6) Implementing AND gate, using a single perceptron

First understand the input data

x1 x2 y 0 0 0 1 0 1 0 1 1

We will represent this in the form of matrix - Rank 2 Tensor Thus we will use the numpy library

```
import numpy as np
#we import the required library

input_data=np.array([[0,0],[0,1],[1,0],[1,1]])
#matrix representation

input_data
array([[0, 0],
       [0, 1],
       [1, 0],
       [1, 1]])

input_data.ndim #its rank
2

input_data.shape
(4, 2)

output_data=np.array([0,0,0,1]) #save the output data

output_data
array([0, 0, 0, 1])

output_data.ndim
1

output_data.shape
(4, )

#now we have to define, what are the additional parameters which are
required
```

$z = x_1w_1 + x_2w_2 + b$

#we randomly initialize the weight, e.g. for the 1st epoch, we initialize as 0

#we can define it individually, but going ahead its not a good practice

`w=[w1,w2]`

#thus we do like above

`weights=np.zeros(2)`

`weights`

`array([0., 0.])`

#above is a method to assign the weights as zero

`weights.ndim`

`1`

`weights.shape`

`(2,)`

#bias is a scalar quantity

`bias = 0`

#the no.of times the perceptron will have a look at the entire dataset is called an epoch

`epoch=10`

#Learning Rate (eta)

`eta=0.1`

#now we define the activation function

```
def activation(z):  
    if z>0:  
        return 1  
    else:  
        return 0
```

`len(input_data)`

`4`

```
for i in range (epoch):  
    print(f'Epoch:{i}')  
    for j in range(len(input_data)):  
        print(input_data[j])
```

Epoch:0

[0 0]

[0 1]

[1 0]

[1 1]

Epoch:1

[0 0]

[0 1]

[1 0]

[1 1]

Epoch:2

[0 0]

[0 1]

[1 0]

[1 1]

Epoch:3

[0 0]

[0 1]

[1 0]

[1 1]

Epoch:4

[0 0]

[0 1]

[1 0]

[1 1]

Epoch:5

[0 0]

[0 1]

[1 0]

[1 1]

Epoch:6

[0 0]

[0 1]

[1 0]

[1 1]

Epoch:7

[0 0]

[0 1]

[1 0]

[1 1]

Epoch:8

[0 0]

[0 1]

[1 0]

[1 1]

```

Epoch:9
[0 0]
[0 1]
[1 0]
[1 1]

for i in range (epoch):
    print(f'Epoch:{i}')
    print()
    for j in range(len(input_data)):
        x=input_data[j]
        yactual=output_data[j]
        z=np.dot(x,weights)+bias
        ypred = activation(z)
        error=yactual-ypred
        # print(error)
        #there is an error in the for iteration, thus we update the
weight
        #refer the notebook
        weights=weights+(eta*error*x)
        bias=bias+eta*error
        #above is the formula to update the bias
        #note: the weights are getting updated after the 1st Epoch

        print("Actual Value: ",yactual)
        print("Predicted Value: ",ypred)
        print("Error: ",error)
        print()
    print()
    print()

```

Epoch:0

```

Actual Value:  0
Predicted Value:  0
Error:  0

```

```

Actual Value:  0
Predicted Value:  0
Error:  0

```

```

Actual Value:  0
Predicted Value:  0
Error:  0

```

```

Actual Value:  1
Predicted Value:  0
Error:  1

```

Epoch:1

Actual Value: 0
Predicted Value: 1
Error: -1

Actual Value: 0
Predicted Value: 1
Error: -1

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 1
Predicted Value: 0
Error: 1

Epoch:2

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 1
Error: -1

Actual Value: 0
Predicted Value: 1
Error: -1

Actual Value: 1
Predicted Value: 0
Error: 1

Epoch:3

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 1
Error: -1

Actual Value: 1
Predicted Value: 0
Error: 1

Epoch:4

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 1
Error: -1

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Epoch:5

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Epoch:6

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Epoch:7

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Epoch:8

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 0

Error: 0

Actual Value: 1

Predicted Value: 1

Error: 0

Epoch:9

Actual Value: 0

Predicted Value: 0

Error: 0

Actual Value: 0

Predicted Value: 0

Error: 0

Actual Value: 0

Predicted Value: 0

Error: 0

Actual Value: 1

Predicted Value: 1

Error: 0

weights

array([0.2, 0.1])

print(bias)

-0.2

7) Observation

Ans:-

- The perceptron was trained using the AND gate dataset for 10 epochs.
- During the initial epochs, prediction errors occurred, causing the weights and bias to be updated.
- By Epoch 5, the perceptron correctly classified all input combinations with zero error.
- The final learned parameters were.
 - Weights $[0.2, 0.1]$
 - Bias -0.2
- The trained perceptron successfully learned the AND gate because it is a linearly separable problem.

8) Result

Ans:-

The Perceptron Learning Algorithm was successfully implemented in Python and tested using the AND gate dataset. After training over multiple epochs, the perceptron learned the correct decision boundary by updating its weights and bias. The trained model correctly classified all four input combinations, ^{confirming} that a single-layer perceptron can successfully solve linearly separable binary classification problems such as the AND gate.

Name: Shardul Narvekar

Roll No.: UAI05BT223414

Division: B

Subject: Deep Learning

Q9.a) Implementing OR gate, using a single perceptron

First understand the input data

x1 x2 y 0 0 0 1 1 1 0 1 1 1 1

We will represent this in the form of matrix - Rank 2 Tensor Thus we will use the numpy library

```
import numpy as np
#we import the required library

input_data=np.array([[0,0],[0,1],[1,0],[1,1]])
#matrix representation

input_data
array([[0, 0],
       [0, 1],
       [1, 0],
       [1, 1]])

input_data.ndim #its rank
2

input_data.shape
(4, 2)

output_data=np.array([0,1,1,1]) #save the output data

output_data
array([0, 1, 1, 1])

output_data.ndim
1

output_data.shape
(4, )

#now we have to define, what are the additional parameters which are
required
```

$z = x_1w_1 + x_2w_2 + b$

#we randomly initialize the weight, e.g. for the 1st epoch, we initialize as 0

#we can define it individually, but going ahead its not a good practice

`w=[w1,w2]`

#thus we do like above

`weights=np.zeros(2)`

`weights`

`array([0., 0.])`

#above is a method to assign the weights as zero

`weights.ndim`

`1`

`weights.shape`

`(2,)`

#bias is a scalar quantity

`bias = 0`

#the no.of times the perceptron will have a look at the entire dataset is called an epoch

`epoch=10`

#Learning Rate (eta)

`eta=0.1`

#now we define the activation function

```
def activation(z):  
    if z>0:  
        return 1  
    else:  
        return 0
```

`len(input_data)`

`4`

```
for i in range (epoch):  
    print(f'Epoch:{i}')  
    for j in range(len(input_data)):  
        print(input_data[j])
```

Epoch:0

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:1

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:2

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:3

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:4

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:5

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:6

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:7

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:8

[0 0]
[0 1]
[1 0]
[1 1]

```

Epoch:9
[0 0]
[0 1]
[1 0]
[1 1]

for i in range (epoch):
    print(f'Epoch:{i}')
    print()
    for j in range(len(input_data)):
        x=input_data[j]
        yactual=output_data[j]
        z=np.dot(x,weights)+bias
        ypred = activation(z)
        error=yactual-ypred
        # print(error)
        #there is an error in the for iteration, thus we update the
weight
        #refer the notebook
        weights=weights+(eta*error*x)
        bias=bias+eta*error
        #above is the formula to update the bias
        #note: the weights are getting updated after the 1st Epoch

        print("Actual Value: ",yactual)
        print("Predicted Value: ",ypred)
        print("Error: ",error)
        print()
    print()
    print()

```

Epoch:0

```

Actual Value:  0
Predicted Value:  0
Error:  0

```

```

Actual Value:  1
Predicted Value:  0
Error:  1

```

```

Actual Value:  1
Predicted Value:  1
Error:  0

```

```

Actual Value:  1
Predicted Value:  1
Error:  0

```

Epoch:1

Actual Value: 0
Predicted Value: 1
Error: -1

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 1
Predicted Value: 1
Error: 0

Epoch:2

Actual Value: 0
Predicted Value: 1
Error: -1

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Epoch:3

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Epoch:4

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Epoch:5

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Epoch:6

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Epoch:7

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Epoch:8

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 1

Error: 0

Actual Value: 1

Predicted Value: 1

Error: 0

Epoch:9

Actual Value: 0

Predicted Value: 0

Error: 0

Actual Value: 1

Predicted Value: 1

Error: 0

Actual Value: 1

Predicted Value: 1

Error: 0

Actual Value: 1

Predicted Value: 1

Error: 0

weights

array([0.1, 0.1])

print(bias)

0.0

Name: Shardul Narvekar

Roll No.: UAI05BT223414

Division: B

Subject: Deep Learning

Q9.b) Implementing NAND gate, using a single perceptron

First understand the input data

x1 x2 y 0 0 0 1 0 1 0 1 1

We will represent this in the form of matrix - Rank 2 Tensor Thus we will use the numpy library

```
import numpy as np
#we import the required library

input_data=np.array([[0,0],[0,1],[1,0],[1,1]])
#matrix representation

input_data
array([[0, 0],
       [0, 1],
       [1, 0],
       [1, 1]])

input_data.ndim #its rank
2

input_data.shape
(4, 2)

output_data=np.array([1,1,1,0]) #save the output data

output_data
array([1, 1, 1, 0])

output_data.ndim
1

output_data.shape
(4, )

#now we have to define, what are the additional parameters which are
required
```

$z = x_1w_1 + x_2w_2 + b$

#we randomly initialize the weight, e.g. for the 1st epoch, we initialize as 0

#we can define it individually, but going ahead its not a good practice

`w=[w1,w2]`

#thus we do like above

`weights=np.zeros(2)`

`weights`

`array([0., 0.])`

#above is a method to assign the weights as zero

`weights.ndim`

`1`

`weights.shape`

`(2,)`

#bias is a scalar quantity

`bias = 0`

#the no.of times the perceptron will have a look at the entire dataset is called an epoch

`epoch=10`

#Learning Rate (eta)

`eta=0.1`

#now we define the activation function

```
def activation(z):  
    if z>0:  
        return 1  
    else:  
        return 0
```

`len(input_data)`

`4`

```
for i in range (epoch):  
    print(f'Epoch:{i}')  
    for j in range(len(input_data)):  
        print(input_data[j])
```

Epoch:0

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:1

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:2

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:3

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:4

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:5

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:6

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:7

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:8

[0 0]
[0 1]
[1 0]
[1 1]

```

Epoch:9
[0 0]
[0 1]
[1 0]
[1 1]

for i in range (epoch):
    print(f'Epoch:{i}')
    print()
    for j in range(len(input_data)):
        x=input_data[j]
        yactual=output_data[j]
        z=np.dot(x,weights)+bias
        ypred = activation(z)
        error=yactual-ypred
        # print(error)
        #there is an error in the for iteration, thus we update the
weight
        #refer the notebook
        weights=weights+(eta*error*x)
        bias=bias+eta*error
        #above is the formula to update the bias
        #note: the weights are getting updated after the 1st Epoch

        print("Actual Value: ",yactual)
        print("Predicted Value: ",ypred)
        print("Error: ",error)
        print()
    print()
    print()

```

Epoch:0

```

Actual Value:  1
Predicted Value:  0
Error:  1

```

```

Actual Value:  1
Predicted Value:  1
Error:  0

```

```

Actual Value:  1
Predicted Value:  1
Error:  0

```

```

Actual Value:  0
Predicted Value:  1
Error:  -1

```

Epoch:1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 0
Predicted Value: 1
Error: -1

Epoch:2

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 0
Predicted Value: 1
Error: -1

Epoch:3

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Epoch:4

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Epoch:5

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Epoch:6

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Epoch:7

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Epoch:8

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 1
Predicted Value: 1

Error: 0

Actual Value: 0

Predicted Value: 0

Error: 0

Epoch:9

Actual Value: 1

Predicted Value: 1

Error: 0

Actual Value: 1

Predicted Value: 1

Error: 0

Actual Value: 1

Predicted Value: 1

Error: 0

Actual Value: 0

Predicted Value: 0

Error: 0

weights

array([-0.2, -0.1])

print(bias)

0.20000000000000004

Name: Shardul Narvekar

Roll No.: UAI05BT223414

Division: B

Subject: Deep Learning

Q9.c) Implementing NOR gate, using a single perceptron

First understand the input data

x1 x2 y 0 0 1 0 1 0 1 0 1 1 0

We will represent this in the form of matrix - Rank 2 Tensor Thus we will use the numpy library

```
import numpy as np
#we import the required library

input_data=np.array([[0,0],[0,1],[1,0],[1,1]])
#matrix representation

input_data
array([[0, 0],
       [0, 1],
       [1, 0],
       [1, 1]])

input_data.ndim #its rank
2

input_data.shape
(4, 2)

output_data=np.array([1,0,0,0]) #save the output data

output_data
array([1, 0, 0, 0])

output_data.ndim
1

output_data.shape
(4, )

#now we have to define, what are the additional parameters which are
required
```

$z = x_1w_1 + x_2w_2 + b$

#we randomly initialize the weight, e.g. for the 1st epoch, we initialize as 0

#we can define it individually, but going ahead its not a good practice

`w=[w1,w2]`

#thus we do like above

`weights=np.zeros(2)`

`weights`

`array([0., 0.])`

#above is a method to assign the weights as zero

`weights.ndim`

`1`

`weights.shape`

`(2,)`

#bias is a scalar quantity

`bias = 0`

#the no.of times the perceptron will have a look at the entire dataset is called an epoch

`epoch=10`

#Learning Rate (eta)

`eta=0.1`

#now we define the activation function

```
def activation(z):  
    if z>0:  
        return 1  
    else:  
        return 0
```

`len(input_data)`

`4`

```
for i in range (epoch):  
    print(f'Epoch:{i}')  
    for j in range(len(input_data)):  
        print(input_data[j])
```

Epoch:0

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:1

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:2

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:3

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:4

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:5

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:6

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:7

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:8

[0 0]
[0 1]
[1 0]
[1 1]

```

Epoch:9
[0 0]
[0 1]
[1 0]
[1 1]

for i in range (epoch):
    print(f'Epoch:{i}')
    print()
    for j in range(len(input_data)):
        x=input_data[j]
        yactual=output_data[j]
        z=np.dot(x,weights)+bias
        ypred = activation(z)
        error=yactual-ypred
        # print(error)
        #there is an error in the for iteration, thus we update the
weight
        #refer the notebook
        weights=weights+(eta*error*x)
        bias=bias+eta*error
        #above is the formula to update the bias
        #note: the weights are getting updated after the 1st Epoch

        print("Actual Value: ",yactual)
        print("Predicted Value: ",ypred)
        print("Error: ",error)
        print()
    print()
    print()

```

Epoch:0

```

Actual Value:  1
Predicted Value:  0
Error:  1

```

```

Actual Value:  0
Predicted Value:  1
Error:  -1

```

```

Actual Value:  0
Predicted Value:  0
Error:  0

```

```

Actual Value:  0
Predicted Value:  0
Error:  0

```

Epoch:1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 1
Error: -1

Actual Value: 0
Predicted Value: 0
Error: 0

Epoch:2

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Epoch:3

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Epoch:4

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Epoch:5

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Epoch:6

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Epoch:7

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Epoch:8

Actual Value: 1
Predicted Value: 1
Error: 0

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 0
Predicted Value: 0

Error: 0

Actual Value: 0

Predicted Value: 0

Error: 0

Epoch:9

Actual Value: 1

Predicted Value: 1

Error: 0

Actual Value: 0

Predicted Value: 0

Error: 0

Actual Value: 0

Predicted Value: 0

Error: 0

Actual Value: 0

Predicted Value: 0

Error: 0

weights

array([-0.1, -0.1])

print(bias)

0.1

Name: Shardul Narvekar

Roll No.: UAI05BT223414

Division: B

Subject: Deep Learning

Q9.d) Implementing XOR gate, using a single perceptron

First understand the input data

x1 x2 y 0 0 0 1 1 1 0 1 1 1 0

We will represent this in the form of matrix - Rank 2 Tensor Thus we will use the numpy library

```
import numpy as np
#we import the required library

input_data=np.array([[0,0],[0,1],[1,0],[1,1]])
#matrix representation

input_data
array([[0, 0],
       [0, 1],
       [1, 0],
       [1, 1]])

input_data.ndim #its rank
2

input_data.shape
(4, 2)

output_data=np.array([0,1,1,0]) #save the output data

output_data
array([0, 1, 1, 0])

output_data.ndim
1

output_data.shape
(4, )

#now we have to define, what are the additional parameters which are
required
```

$z = x_1w_1 + x_2w_2 + b$

#we randomly initialize the weight, e.g. for the 1st epoch, we initialize as 0

#we can define it individually, but going ahead its not a good practice

`w=[w1,w2]`

#thus we do like above

`weights=np.zeros(2)`

`weights`

`array([0., 0.])`

#above is a method to assign the weights as zero

`weights.ndim`

`1`

`weights.shape`

`(2,)`

#bias is a scalar quantity

`bias = 0`

#the no.of times the perceptron will have a look at the entire dataset is called an epoch

`epoch=10`

#Learning Rate (eta)

`eta=0.1`

#now we define the activation function

```
def activation(z):  
    if z>0:  
        return 1  
    else:  
        return 0
```

`len(input_data)`

`4`

```
for i in range (epoch):  
    print(f'Epoch:{i}')  
    for j in range(len(input_data)):  
        print(input_data[j])
```

Epoch:0

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:1

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:2

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:3

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:4

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:5

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:6

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:7

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:8

[0 0]
[0 1]
[1 0]
[1 1]

```

Epoch:9
[0 0]
[0 1]
[1 0]
[1 1]

for i in range (epoch):
    print(f'Epoch:{i}')
    print()
    for j in range(len(input_data)):
        x=input_data[j]
        yactual=output_data[j]
        z=np.dot(x,weights)+bias
        ypred = activation(z)
        error=yactual-ypred
        # print(error)
        #there is an error in the for iteration, thus we update the
weight
        #refer the notebook
        weights=weights+(eta*error*x)
        bias=bias+eta*error
        #above is the formula to update the bias
        #note: the weights are getting updated after the 1st Epoch

        print("Actual Value: ",yactual)
        print("Predicted Value: ",ypred)
        print("Error: ",error)
        print()
    print()
    print()

```

Epoch:0

```

Actual Value:  0
Predicted Value:  0
Error:  0

```

```

Actual Value:  1
Predicted Value:  0
Error:  1

```

```

Actual Value:  1
Predicted Value:  1
Error:  0

```

```

Actual Value:  0
Predicted Value:  1
Error:  -1

```

Epoch:1

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 0
Predicted Value: 1
Error: -1

Epoch:2

Actual Value: 0
Predicted Value: 1
Error: -1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 0
Predicted Value: 1
Error: -1

Epoch:3

Actual Value: 0
Predicted Value: 1
Error: -1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 0
Predicted Value: 1
Error: -1

Epoch:4

Actual Value: 0
Predicted Value: 1
Error: -1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 0
Predicted Value: 1
Error: -1

Epoch:5

Actual Value: 0
Predicted Value: 1
Error: -1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 0
Predicted Value: 1
Error: -1

Epoch:6

Actual Value: 0
Predicted Value: 1
Error: -1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 0
Predicted Value: 1
Error: -1

Epoch:7

Actual Value: 0
Predicted Value: 1
Error: -1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 0
Predicted Value: 1
Error: -1

Epoch:8

Actual Value: 0
Predicted Value: 1
Error: -1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 1
Predicted Value: 0

Error: 1

Actual Value: 0

Predicted Value: 1

Error: -1

Epoch:9

Actual Value: 0

Predicted Value: 1

Error: -1

Actual Value: 1

Predicted Value: 0

Error: 1

Actual Value: 1

Predicted Value: 0

Error: 1

Actual Value: 0

Predicted Value: 1

Error: -1

weights

array([-0.1, 0.])

print(bias)

0.1

10) Is the Perceptron able to learn the XOR function? Explain why?
Ans:-

No, a single-layer perceptron is not able to learn the XOR function because the XOR dataset is not linearly separable. A single perceptron can only classify data that can be separated by a single straight line. ~~For~~ Since the XOR inputs cannot be separated by a single decision boundary, the perceptron fails to converge and continuously produces classification errors even after multiple epochs. Therefore, solving the XOR problem requires a Multi-Layer Perceptron (MLP) with one or more hidden layers and non-linear activation functions.